Laravel + Inertia.js + React Portfolio Activity

Student: Mark Kirby Benitez
Course and section: BSIT 3, Block 5
Date: November 2, 2026
Dynamic section: Projects

Run the application

Install the project's Composer and npm dependencies, then run php artisan migrate and composer run dev. Register an account to sign in, open Manage or /projects, and add a real project so it appears in the public portfolio. Run the full test suite with php artisan test; regenerate this report and its source appendix with npm run docs:pdf.

Section and justification

I chose Projects because project entries change as I complete school and personal work. Storing them in the database means I can maintain the portfolio from an authenticated form without editing or rebuilding the React page. The public page reads the saved project records and still displays the portfolio when no records have been added.

Fields

Field	Database type	Purpose
`title`	String, maximum 120 characters, unique	Identifies a project and prevents accidental duplicate entries.
`description`	Text, maximum 1,000 characters	Explains what the project does.
`category`	String, limited to `school`, `personal`, or `planned` by validation	Supports portfolio filtering.
`technologies`	String, maximum 255 characters	Lists the tools and technologies used.
`url`	Nullable string, maximum 2,048 characters	Optional link to a live project.

Laravel also adds an integer primary key and created_at / updated_at timestamps. The five fields above are the meaningful project fields.

Validation rules

Validation lives in app/Http/Requests/ProjectRequest.php, so every create and update request is checked on the server:

- title is required, a string, at most 120 characters, and unique. On update, the current project is excluded from the uniqueness check.
- description is required, a string, and at most 1,000 characters.
- category is required and must be one of the three supported categories.
- technologies is required, a string, and at most 255 characters.
- url may be blank, but when supplied it must be a valid URL and at most 2,048 characters.
- Custom messages explain duplicate titles and malformed URLs in plain language.

The browser form also provides helpful input constraints, but Laravel remains authoritative because client-side rules can be bypassed.

Request flow and design decisions

- routes/web.php registers the resource routes inside auth middleware. Guests are sent to login before they can manage projects.
- ProjectController performs one focused action for each page or mutation. It queries records, renders an Inertia page, or redirects after a validated write.
- ProjectRequest authorizes signed-in users and applies the same validation rules to both store and update.
- The controller passes only $request->validated() to Project::create() / update(). The model's $fillable list is the explicit mass-assignment allowlist.
- Inertia renders the React page named by the controller. useForm sends requests, keeps typed values when validation fails, and exposes Laravel's field errors beside their inputs.
- On success, the controller redirects with a one-request flash message. The Inertia middleware shares that message with React, where the index page announces it.
- The public / route queries the same table and passes projects to the portfolio React page, which renders cards and filters by category.

A separate resource controller keeps HTTP concerns out of the route file and gives each CRUD action a conventional URL and name. The show route is omitted because projects are displayed as cards on the public portfolio and the manager index already lists each record. ProjectForm is shared between Create and Edit to keep field behavior, validation display, and styling consistent.

Resource routes used

Method and URL	Name	Action

`GET /projects` | `projects.index` | List records and show success feedback.
`GET /projects/create` | `projects.create` | Render the create form.
`POST /projects` | `projects.store` | Validate and create a record.
`GET /projects/{project}/edit` | `projects.edit` | Render the edit form.
`PUT/PATCH /projects/{project}` | `projects.update` | Validate and update a record.
`DELETE /projects/{project}` | `projects.destroy` | Delete after browser confirmation.

If $fillable were removed without adding a $guarded policy, Eloquent would normally discard mass-assigned attributes (or throw in strict mode), so the submitted project fields would not be saved. The allowlist makes the fields this form may set explicit and prevents unrelated model fields from being mass-assigned.

Validation test table

Run with php artisan test tests/Feature/ProjectManagementTest.php. The feature tests use an in-memory SQLite database and exercise the server rules and redirect responses.

#	Test input	Expected result	Actual result	Pass / fail
1	Submit an empty create request.	Errors for title, description, category, and technologies; no project is stored.	Required-field errors were returned and the projects table remained empty.	Pass
2	Submit a title longer than 120 characters.	A title length validation error; no record is created.	The title error was returned.	Pass
3	Submit a description longer than 1,000 characters and technologies longer than 255 characters.	Errors appear for both fields.	Both field errors were returned.	Pass
4	Submit an unsupported category and malformed project URL.	Errors appear for category and URL, including the custom URL message.	Both fields were rejected by validation.	Pass
5	Create a second project with an existing title.	The duplicate title is rejected with a custom message.	"A project with this title already exists." was returned.	Pass
6	Update a project without changing its title.	Update succeeds because the unique rule ignores the current row.	The project saved with the same title and updated description.	Pass

Validation failures redirect back with field errors and flashed input. Inertia's useForm retains the component form state on an error response, so values are not cleared while the user corrects a field.

Screenshots to include in the submitted PDF

Capture these screens from the running app and label each figure. The management screens require a registered, logged-in user.

- Public portfolio with at least one database-backed project. The current capture is docs/screenshots/public-portfolio.png and shows the database-backed Projects empty state; add an actual project and replace this capture for submission.
- Project manager index.
- Create project page.
- Edit project page with existing values.
- Delete confirmation dialog.
- Success message after saving or deleting.
- Validation errors shown next to fields.
- File/folder structure showing the migration, model, request, controller, routes, pages, and shared form.

Code map and explanations

The complete implementation is in the files below. Include their source in the final PDF appendix.

File	Explanation
`database/migrations/2026_11_01_000000_create_projects_table.php`	Defines the five project fields and timestamps.
`database/migrations/2026_11_02_000001_add_unique_title_to_projects_table.php`	Adds a database-level unique index to reinforce application validation.
`app/Models/Project.php`	Defines the Eloquent model and the safe mass-assignment fields.
`app/Http/Requests/ProjectRequest.php`	Centralizes authorization, field validation, update-aware uniqueness, and

custom messages.
`app/Http/Controllers/ProjectController.php` | Handles index/create/edit rendering and validated create/update/delete operations.
`routes/web.php` | Exposes the public portfolio and authenticated resource routes.
`resources/js/components/projects/project-form.tsx` | Reusable form for creating and editing, including inline field errors.
`resources/js/pages/projects/index.tsx` | Lists projects, links to create/edit, confirms delete, and displays success feedback.
`resources/js/pages/projects/create.tsx` | Wraps the shared form in the authenticated app layout.
`resources/js/pages/projects/edit.tsx` | Passes the selected project into the shared form.
`resources/js/pages/welcome.tsx` | Renders the public portfolio and database-backed filtered project cards.
`app/Http/Middleware/HandleInertiaRequests.php` | Shares authentication and flash success data with Inertia pages.

Video demonstration outline

The video must be screen-recorded with the student's own narration; it has not been recorded or uploaded as part of this source-code task. Record it continuously and add the resulting shareable Google Drive or unlisted YouTube URL to the submitted report.

- Introduce yourself, the portfolio, and why Projects is the dynamic section.
- Show the migration, model, request, controller, routes, React pages, and shared form.
- Explain Route ? Controller ? Form Request validation ? Model ? Inertia props/redirect ? React.
- Create, view on the public page, edit, and delete a record.
- Trigger empty-field, malformed URL, and duplicate-title errors.
- Without stopping the recording, add a new field end-to-end (migration, model allowlist, validation, form, public display), migrate, and demonstrate it.
- Answer why the controller, Route::resource, $fillable, server validation, Inertia errors, and shared form are used.

Reflection prompts

These are reflection drafts based on the implementation. Personalize them to match what you experienced before submitting:

What was hardest about converting the static page, and how did you solve it?
The hardest part was moving the page into the Laravel/Inertia entry point without losing the original theme, navigation, and responsive styling. I rebuilt the page as React components, moved the theme variables into the Tailwind CSS entry point, and used the public route to pass database projects into the page.

What did you learn about server-side validation?
I learned that browser validation is only a convenience because a request can be sent without using the form. Laravel validates every create and update request, returns field errors through Inertia, and preserves the form state so the user can correct the values. I also learned that update-time unique validation must ignore the current row.

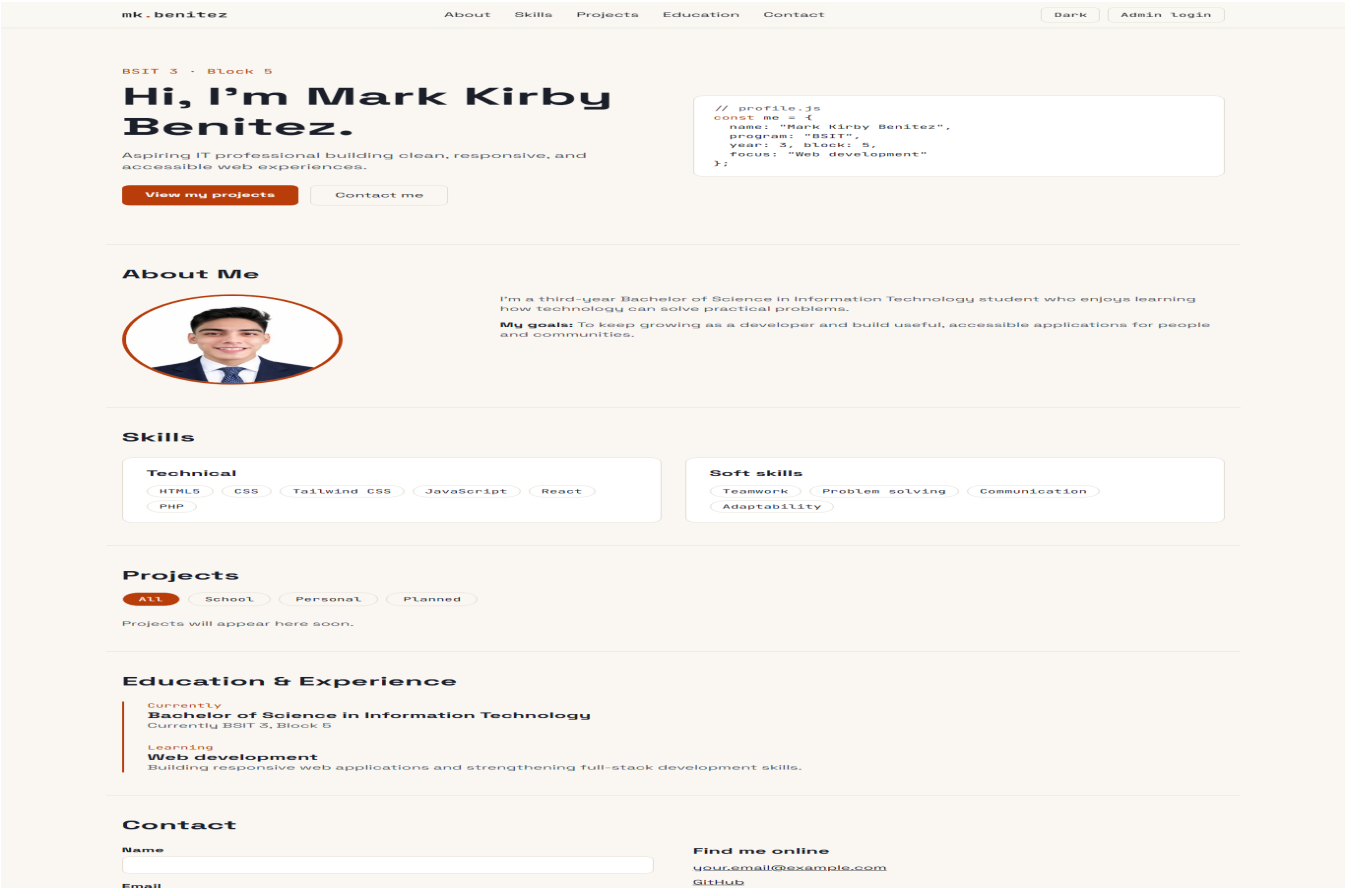
What would you change when adding another dynamic section?
I would first define the fields, database constraints, validation rules, and public display requirements together. I would then reuse the existing patterns for a Form Request, resource controller, shared form, flash feedback, and feature tests, while keeping the new section's data model and validation independent.

Repository and video links

- Repository: add the submitted GitHub URL here.
- Demonstration video: add the shareable Google Drive or unlisted YouTube URL here.

FIGURE: PUBLIC PORTFOLIO

The public Projects section is database-backed and filterable. This capture shows the current empty state; add a real project before submitting a populated portfolio screenshot.



Captured from the running Laravel + Inertia application on November 2, 2026.
The manager, create, edit, confirmation, success, and validation screenshots still need to be added from an authenticated browser session.

APPENDIX: COMPLETE SOURCE CODE

The following source files are reproduced from the project.

database/migrations/2026_11_01_000000_create_projects_table.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration
{
public function up(): void
{
Schema::create('projects', function (Blueprint $table) {
$table->id();
$table->string('title', 120);
$table->text('description');
$table->string('category', 20);
$table->string('technologies', 255);
$table->string('url', 2048)->nullable();
$table->timestamps();
});
}
public function down(): void
{
Schema::dropIfExists('projects');
}
};
```

database/migrations/2026_11_02_000001_add_unique_title_to_projects_table.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration
{
public function up(): void
{
Schema::table('projects', function (Blueprint $table) {
$table->unique('title');
});
}
public function down(): void
{
Schema::table('projects', function (Blueprint $table) {
$table->dropUnique(['title']);
});
}
};
```

app/Models/Project.php

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Project extends Model
{
protected $fillable = [
'title',
'description',
'category',
'technologies',
'url',
];
}
```

app/Http/Requests/ProjectRequest.php

```
<?php
namespace App\Http\Requests;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
class ProjectRequest extends FormRequest
{
public function authorize(): bool
{
return $this->user() !== null;
}
public function rules(): array
{
$project = $this->route('project');
return [
'title' => [
'required',
'string',
'max:120',
Rule::unique('projects', 'title')->ignore($project),
],
'description' => ['required', 'string', 'max:1000'],
'category' => ['required', Rule::in(['school', 'personal', 'planned'])],
'technologies' => ['required', 'string', 'max:255'],
'url' => ['nullable', 'url', 'max:2048'],
];
}
public function messages(): array
{
return [
'title.unique' => 'A project with this title already exists.',
'url.url' => 'Enter a complete project URL, including https://.',
];
}
}
```

app/Http/Controllers/ProjectController.php

```
<?php
namespace App\Http\Controllers;
use App\Http\Requests\ProjectRequest;
use App\Models\Project;
use Illuminate\Http\RedirectResponse;
use Inertia\Inertia;
use Inertia\Response;
class ProjectController extends Controller
{
public function index(): Response
{
return Inertia::render('projects/index', [
'projects' => Project::query()->latest()->get(),
]);
}
public function create(): Response
{
return Inertia::render('projects/create');
}
public function edit(Project $project): Response
{
return Inertia::render('projects/edit', [
'project' => $project,
]);
}
public function store(ProjectRequest $request): RedirectResponse
{
Project::query()->create($request->validated());
return to_route('projects.index')->with('success', 'Project created.');
}
public function update(ProjectRequest $request, Project $project): RedirectResponse
{
$project->update($request->validated());
return to_route('projects.index')->with('success', 'Project updated.');
}
public function destroy(Project $project): RedirectResponse
{
$project->delete();
return to_route('projects.index')->with('success', 'Project deleted.');
}
}
```

routes/web.php

```
<?php
use App\Http\Controllers\ProjectController;
use App\Models\Project;
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;
Route::get('/', function () {
return Inertia::render('welcome', [
'projects' => Project::query()->latest()->get(),
]);
})->name('home');
Route::middleware(['auth'])->group(function () {
Route::resource('projects', ProjectController::class)->except(['show']);
Route::get('dashboard', function () {
return Inertia::render('dashboard');
})->name('dashboard');
});
require __DIR__.'/settings.php';
require __DIR__.'/auth.php';
```

app/Http/Middleware/HandleInertiaRequests.php

```
<?php
namespace App\Http\Middleware;
use Illuminate\Foundation\Inspiring;
use Illuminate\Http\Request;
use Inertia\Middleware;
class HandleInertiaRequests extends Middleware
{
/**
* The root template that's loaded on the first page visit.
*
* @see https://inertiajs.com/server-side-setup#root-template
*
* @var string
*/
protected $rootView = 'app';
/**
* Determines the current asset version.
*
* @see https://inertiajs.com/asset-versioning
*/
public function version(Request $request): ?string
{
return parent::version($request);
}
/**
* Define the props that are shared by default.
*
* @see https://inertiajs.com/shared-data
*
* @return array<string, mixed>
*/
public function share(Request $request): array
{
[$message, $author] = str(Inspiring::quotes()->random())->explode('-');
return array_merge(parent::share($request), [
...parent::share($request),
'name' => config('app.name'),
'quote' => ['message' => trim($message), 'author' => trim($author)],
'auth' => [
'user' => $request->user(),
],
'flash' => [
'success' => fn () => $request->session()->get('success'),
],
]);
}
}
```

resources/js/components/projects/project-form.tsx

```
import { Input } from '@/components/ui/input';
import { Head, Link, useForm } from '@inertiajs/react';
import { type FormEvent } from 'react';
export type Project = {
id: number;
title: string;
description: string;
category: 'school' | 'personal' | 'planned';
technologies: string;
url: string | null;
};
type ProjectData = Omit<Project, 'id'>;
const categories: ProjectData['category'][] = ['school', 'personal', 'planned'];
export default function ProjectForm({ project }: { project?: Project }) {
const form = useForm<ProjectData>({
title: project?.title ?? '',
description: project?.description ?? '',
category: project?.category ?? 'school',
technologies: project?.technologies ?? '',
url: project?.url ?? '',
});
function submit(event: FormEvent<HTMLFormElement>) {
event.preventDefault();
const options = {
onSuccess: () => form.reset(),
};
if (project) {
form.put(`/projects/${project.id}`, options);
} else {
form.post('/projects', options);
}
}
return (
<>
<Head title={project ? 'Edit Project' : 'Create Project'} />
<div className="mx-auto w-full max-w-2xl p-4">
<div className="mb-6 flex items-center justify-between gap-4">
<div>
<h1 className="text-2xl font-semibold">{project ? 'Edit project' : 'Create a project'}</h1>
<p className="text-muted-foreground mt-1 text-sm">Fields marked required must be filled in.</p>
</div>
<Link href="/projects" className="text-sm underline underline-offset-4">
Back to projects
</Link>
</div>
<form onSubmit={submit} className="border-sidebar-border/70 dark:border-sidebar-border space-y-5 rounded-xl border
p-6">
<div>
<label htmlFor="title" className="mb-1 block text-sm font-medium">
Title <span aria-hidden="true">*</span>
</label>
<Input
id="title"
value={form.data.title}
onChange={(event) => form.setData('title', event.target.value)}
maxLength={120}
required
aria-invalid={Boolean(form.errors.title)}
aria-describedby={form.errors.title ? 'title-error' : undefined}
/>
{form.errors.title && (
<p id="title-error" className="text-destructive mt-1 text-sm" role="alert">
{form.errors.title}
</p>
)}
</div>
<div>
<label htmlFor="description" className="mb-1 block text-sm font-medium">
Description <span aria-hidden="true">*</span>
</label>
<textarea
id="description"
rows={5}
value={form.data.description}
onChange={(event) => form.setData('description', event.target.value)}
maxLength={1000}
```

```
required
className="border-input bg-background focus-visible:ring-ring w-full rounded-md border px-3 py-2 text-sm
focus-visible:ring-2 focus-visible:outline-hidden"
aria-invalid={Boolean(form.errors.description)}
aria-describedby={form.errors.description ? 'description-error' : undefined}
/>
{form.errors.description && (
<p id="description-error" className="text-destructive mt-1 text-sm" role="alert">
{form.errors.description}
</p>
)}
</div>
<div>
<label htmlFor="category" className="mb-1 block text-sm font-medium">
Category <span aria-hidden="true">*</span>
</label>
<select
id="category"
value={form.data.category}
onChange={(event) => {
const category = categories.find((option) => option === event.target.value);
if (category) form.setData('category', category);
}}
required
className="border-input bg-background w-full rounded-md border px-3 py-2 text-sm"
aria-invalid={Boolean(form.errors.category)}
aria-describedby={form.errors.category ? 'category-error' : undefined}
>
{categories.map((category) => (
<option key={category} value={category}>
{category}
</option>
))}
</select>
{form.errors.category && (
<p id="category-error" className="text-destructive mt-1 text-sm" role="alert">
{form.errors.category}
</p>
)}
</div>
<div>
<label htmlFor="technologies" className="mb-1 block text-sm font-medium">
Technologies <span aria-hidden="true">*</span>
</label>
<Input
id="technologies"
value={form.data.technologies}
onChange={(event) => form.setData('technologies', event.target.value)}
placeholder="React, Laravel, Tailwind CSS"
maxLength={255}
required
aria-invalid={Boolean(form.errors.technologies)}
aria-describedby={form.errors.technologies ? 'technologies-error' : undefined}
/>
{form.errors.technologies && (
<p id="technologies-error" className="text-destructive mt-1 text-sm" role="alert">
{form.errors.technologies}
</p>
)}
</div>
<div>
<label htmlFor="url" className="mb-1 block text-sm font-medium">
Project URL <span className="text-muted-foreground">(optional)</span>
</label>
<Input
id="url"
type="url"
value={form.data.url ?? ''}
onChange={(event) => form.setData('url', event.target.value)}
maxLength={2048}
aria-invalid={Boolean(form.errors.url)}
aria-describedby={form.errors.url ? 'url-error' : undefined}
/>
{form.errors.url && (
<p id="url-error" className="text-destructive mt-1 text-sm" role="alert">
{form.errors.url}
</p>
)}
```

```
</div>
<div className="flex flex-wrap gap-3">
<button
type="submit"
disabled={form.processing}
className="bg-primary text-primary-foreground rounded-md px-4 py-2 text-sm font-medium disabled:opacity-50"
>
{form.processing ? 'Saving...' : project ? 'Save changes' : 'Create project'}
</button>
<Link href="/projects" className="border-input rounded-md border px-4 py-2 text-sm">
Cancel
</Link>
</div>
</form>
</div>
</>
);
}
```

resources/js/components/projects/project-card.tsx

```
import { type Project } from '@/components/projects/project-form';
export default function ProjectCard({ project }: { project: Project }) {
return (
<article className="border-line bg-surface flex flex-col rounded-xl border p-6">
<h3 className="text-xl font-bold">{project.title}</h3>
<p className="text-muted mt-2 grow">{project.description}</p>
<p className="mt-4 font-mono text-sm">{project.technologies}</p>
{project.url && (
<a href={project.url} target="_blank" rel="noopener noreferrer" className="text-accent mt-4 font-semibold
underline">
View project
</a>
)}
</article>
);
}
```

resources/js/components/portfolio-navbar.tsx

```
import { Link } from '@inertiajs/react';
import { useState } from 'react';
type Props = {
authenticated: boolean;
theme: 'light' | 'dark';
onThemeChange: () => void;
};
const links = [
['About', 'about'],
['Skills', 'skills'],
['Projects', 'projects'],
['Education', 'education'],
['Contact', 'contact'],
];
export default function PortfolioNavbar({ authenticated, theme, onThemeChange }: Props) {
const [menuOpen, setMenuOpen] = useState(false);
return (
<header className="border-line bg-bg/90 sticky top-0 z-40 border-b backdrop-blur">
<nav className="relative mx-auto flex max-w-6xl items-center justify-between px-4 py-3" aria-label="Main
navigation">
<a href="#home" className="font-mono text-lg font-semibold">
mk<span className="text-accent">.</span>benitez
</a>
<ul
id="nav-menu"
className={`${menuOpen ? 'flex' : 'hidden'} border-line bg-bg absolute inset-x-0 top-full flex-col gap-1 border-b
p-4 md:static md:flex md:flex-row md:gap-6 md:border-0 md:p-0`}
>
{links.map(([label, id]) => (
<li key={id}>
<a className="hover:text-accent block py-2" href={`#${id}`} onClick={() => setMenuOpen(false)}>
{label}
</a>
</li>
))}
</ul>
<div className="flex items-center gap-2">
<button
type="button"
className="border-line hover:border-accent rounded-lg border px-3 py-2 font-mono text-sm"
aria-label={`Switch to ${theme === 'dark' ? 'light' : 'dark'} mode`}
onClick={onThemeChange}
>
{theme === 'dark' ? 'Light' : 'Dark'}
</button>
<Link
href={authenticated ? '/projects' : '/login'}
className="border-line hover:border-accent hidden rounded-lg border px-3 py-2 font-mono text-sm sm:block"
>
{authenticated ? 'Manage' : 'Admin login'}
</Link>
<button
id="menu-toggle"
type="button"
className="border-line rounded-lg border px-3 py-2 font-mono text-sm md:hidden"
aria-expanded={menuOpen}
aria-controls="nav-menu"
onClick={() => setMenuOpen(!menuOpen)}
>
Menu
</button>
</div>
</nav>
</header>
);
}
```

resources/js/components/portfolio-footer.tsx

```
export default function PortfolioFooter() {
return (
<footer className="border-line text-muted border-t py-8 text-center text-sm">
<p>&copy; 2026 Mark Kirby Benitez &middot; BSIT 3 Block 5 &middot; Built with Laravel, Inertia.js, React, and
Tailwind CSS.</p>
</footer>
);
}
```

resources/js/components/section-title.tsx

```
type Props = {
children: string;
className?: string;
};
export default function SectionTitle({ children, className = 'mb-8' }: Props) {
return <h2 className={`text-3xl font-bold ${className}`}>{children}</h2>;
}
```

resources/js/pages/projects/index.tsx

```
import { type Project } from '@/components/projects/project-form';
import AppLayout from '@/layouts/app-layout';
import { type BreadcrumbItem, type SharedData } from '@/types';
import { Head, Link, router, usePage } from '@inertiajs/react';
const breadcrumbs: BreadcrumbItem[] = [{ title: 'Projects', href: '/projects' }];
export default function ProjectsIndex({ projects }: { projects: Project[] }) {
const { flash } = usePage<SharedData>().props;
function remove(project: Project) {
if (!window.confirm(`Delete "${project.title}"? This cannot be undone.`)) return;
router.delete(`/projects/${project.id}`);
}
return (
<AppLayout breadcrumbs={breadcrumbs}>
<Head title="Manage Projects" />
<div className="flex flex-1 flex-col gap-6 p-4">
<div className="flex flex-wrap items-center justify-between gap-3">
<div>
<h1 className="text-2xl font-semibold">Manage portfolio projects</h1>
<p className="text-muted-foreground mt-1 text-sm">Add, edit, or remove projects shown on your public portfolio.</p>
</div>
<div className="flex items-center gap-4">
<Link href="/" className="text-sm underline underline-offset-4">
View portfolio
</Link>
<Link href="/projects/create" className="bg-primary text-primary-foreground rounded-md px-4 py-2 text-sm
font-medium">
Add project
</Link>
</div>
</div>
{flash.success && (
<p
className="rounded-md border border-green-600/30 bg-green-600/10 px-4 py-3 text-sm text-green-800
dark:text-green-300"
role="status"
>
{flash.success}
</p>
)}
{projects.length === 0 ? (
<p className="border-sidebar-border/70 text-muted-foreground dark:border-sidebar-border rounded-xl border
border-dashed p-6 text-sm">
No projects yet. Add your first project to display it on your public portfolio.
</p>
) : (
<div className="space-y-3">
{projects.map((project) => (
<article key={project.id} className="border-sidebar-border/70 dark:border-sidebar-border rounded-xl border p-5">
<div className="flex flex-wrap items-start justify-between gap-4">
<div className="min-w-0">
<h2 className="font-semibold">{project.title}</h2>
<p className="text-muted-foreground mt-1 text-xs capitalize">
{project.category} &middot; {project.technologies}
</p>
<p className="text-muted-foreground mt-3 text-sm">{project.description}</p>
{project.url && (
<a
className="mt-2 inline-block text-sm underline"
href={project.url}
target="_blank"
rel="noopener noreferrer"
>
Project link
</a>
)}
</div>
<div className="flex shrink-0 gap-2">
<Link href={`/projects/${project.id}/edit`} className="border-input rounded-md border px-3 py-1.5 text-sm">
Edit
</Link>
<button
type="button"
onClick={() => remove(project)}
className="border-destructive text-destructive rounded-md border px-3 py-1.5 text-sm"
>
Delete
```

```
</button>
</div>
</div>
</article>
))}
</div>
)}
</div>
</AppLayout>
);
}
```

resources/js/pages/projects/create.tsx

```
import ProjectForm from '@/components/projects/project-form';
import AppLayout from '@/layouts/app-layout';
import { type BreadcrumbItem } from '@/types';
const breadcrumbs: BreadcrumbItem[] = [
{ title: 'Projects', href: '/projects' },
{ title: 'Create', href: '/projects/create' },
];
export default function CreateProject() {
return (
<AppLayout breadcrumbs={breadcrumbs}>
<ProjectForm />
</AppLayout>
);
}
```

resources/js/pages/projects/edit.tsx

```
import ProjectForm, { type Project } from '@/components/projects/project-form';
import AppLayout from '@/layouts/app-layout';
import { type BreadcrumbItem } from '@/types';
export default function EditProject({ project }: { project: Project }) {
const breadcrumbs: BreadcrumbItem[] = [
{ title: 'Projects', href: '/projects' },
{ title: 'Edit', href: `/projects/${project.id}/edit` },
];
return (
<AppLayout breadcrumbs={breadcrumbs}>
<ProjectForm project={project} />
</AppLayout>
);
}
```

resources/js/pages/welcome.tsx

```
import PortfolioFooter from '@/components/portfolio-footer';
import PortfolioNavbar from '@/components/portfolio-navbar';
import ProjectCard from '@/components/projects/project-card';
import { type Project } from '@/components/projects/project-form';
import SectionTitle from '@/components/section-title';
import { Head } from '@inertiajs/react';
import { useEffect, useState, type FormEvent } from 'react';
type Props = {
projects: Project[];
auth: { user: { id: number } | null };
};
const filters = ['all', 'school', 'personal', 'planned'] as const;
type ProjectFilter = (typeof filters)[number];
export default function Welcome({ projects, auth }: Props) {
const [activeFilter, setActiveFilter] = useState<ProjectFilter>('all');
const [theme, setTheme] = useState<'light' | 'dark'>('light');
const [themeReady, setThemeReady] = useState(false);
const [contactStatus, setContactStatus] = useState('');
const visibleProjects = projects.filter((project) => activeFilter === 'all' || project.category === activeFilter);
useEffect(() => {
let savedTheme: string | null = null;
try {
savedTheme = window.localStorage.getItem('appearance');
} catch {
savedTheme = null;
}
setTheme(
savedTheme === 'dark' || savedTheme === 'light'
? savedTheme
: window.matchMedia('(prefers-color-scheme: dark)').matches
? 'dark'
: 'light',
);
setThemeReady(true);
}, []);
useEffect(() => {
if (!themeReady) return;
document.documentElement.classList.toggle('dark', theme === 'dark');
try {
window.localStorage.setItem('appearance', theme);
} catch {
// Keep the selected theme for this page even when storage is unavailable.
}
}, [theme, themeReady]);
function submitContact(event: FormEvent<HTMLFormElement>) {
event.preventDefault();
setContactStatus('Thank you! Your message was validated (demo only, nothing was sent).');
event.currentTarget.reset();
}
return (
<>
<Head title="Mark Kirby Benitez | BSIT Portfolio">
<meta name="description" content="Portfolio of Mark Kirby Benitez, BSIT 3 student and aspiring IT professional." />
</Head>
<div className="bg-bg text-ink min-h-screen font-sans antialiased">
<a href="#main" className="bg-accent text-bg sr-only fixed top-2 left-2 z-50 rounded px-4 py-2 focus:not-sr-only">
Skip to content
</a>
<PortfolioNavbar
authenticated={Boolean(auth.user)}
theme={theme}
onThemeChange={() => setTheme(theme === 'dark' ? 'light' : 'dark')}
/>
<main id="main">
<section id="home" className="mx-auto grid max-w-6xl items-center gap-10 px-4 py-16 md:grid-cols-2 md:py-24">
<div>
<p className="text-accent mb-3 font-mono">BSIT 3 &middot; Block 5</p>
<h1 className="text-4xl leading-tight font-bold sm:text-5xl lg:text-6xl">Hi, I&apos;m Mark Kirby Benitez.</h1>
<p className="text-muted mt-5 max-w-prose text-lg">
Aspiring IT professional building clean, responsive, and accessible web experiences.
</p>
<div className="mt-8 flex flex-wrap gap-3">
<a href="#projects" className="bg-accent text-bg rounded-lg px-6 py-3 font-semibold hover:opacity-90">
View my projects
</a>
<a href="#contact" className="border-line hover:border-accent rounded-lg border px-6 py-3">
```

```
Contact me
</a>
</div>
</div>
<div
className="border-line bg-surface rounded-xl border p-5 font-mono text-sm leading-relaxed"
aria-label="Profile summary card"
>
<p className="text-muted">// profile.js</p>
<p>
<span className="text-accent">const</span> me = {'{'}
</p>
<p className="pl-4">name: &amp;quot;Mark Kirby Benitez&amp;quot;,</p>
<p className="pl-4">program: &amp;quot;BSIT&amp;quot;,</p>
<p className="pl-4">year: 3, block: 5,</p>
<p className="pl-4">focus: &amp;quot;Web development&amp;quot;</p>
<p>{'}'};</p>
</div>
</section>
<section id="about" className="border-line mx-auto max-w-6xl border-t px-4 py-14">
<SectionTitle>About Me</SectionTitle>
<div className="grid items-start gap-8 md:grid-cols-3">
<div className="mx-auto md:mx-0">
<img
src="/images/me.jpg"
alt="Portrait photo of Mark Kirby Benitez in a navy suit and polka-dot tie"
width="600"
height="600"
className="border-accent h-48 w-48 rounded-full border-4 object-cover md:h-56 md:w-56"
/>
</div>
<div className="text-muted space-y-4 md:col-span-2">
<p>
I&amp;apos;m a third-year Bachelor of Science in Information Technology student who enjoys learning how technology can
solve practical problems.
</p>
<p>
<strong className="text-ink">My goals:</strong> To keep growing as a developer and build useful, accessible
applications for people and communities.
</p>
</div>
</div>
</section>
<section id="skills" className="border-line mx-auto max-w-6xl border-t px-4 py-14">
<SectionTitle>Skills</SectionTitle>
<div className="grid gap-6 md:grid-cols-2">
{[
{ title: 'Technical', skills: ['HTML5', 'CSS', 'Tailwind CSS', 'JavaScript', 'React', 'PHP'] },
{ title: 'Soft skills', skills: ['Teamwork', 'Problem solving', 'Communication', 'Adaptability'] },
].map(({ title, skills }) => (
<article key={title} className="border-line bg-surface rounded-xl border p-6">
<h3 className="mb-4 text-xl font-bold">{title}</h3>
<ul className="flex flex-wrap gap-2 font-mono text-sm">
{skills.map((skill) => (
<li key={skill} className="border-line rounded-full border px-3 py-1">
{skill}
</li>
))}
</ul>
</article>
))}
</div>
</section>
<section id="projects" className="border-line mx-auto max-w-6xl border-t px-4 py-14">
<SectionTitle className="mb-6">Projects</SectionTitle>
<div className="mb-8 flex flex-wrap gap-2" role="group" aria-label="Filter projects by category">
{filters.map((filter) => (
<button
key={filter}
type="button"
className={`border-line rounded-full border px-4 py-1.5 font-mono text-sm capitalize ${activeFilter === filter ?
'bg-accent text-bg' : ''}`}
aria-pressed={activeFilter === filter}
onClick={() => setActiveFilter(filter)}
>
{filter}
</button>
))}
```

```
</div>
<div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-3">
{visibleProjects.map((project) => (
<ProjectCard key={project.id} project={project} />
))}
{visibleProjects.length === 0 && (
<p className="text-muted">
{projects.length === 0 ? 'Projects will appear here soon.' : 'No projects in this category yet.'}
</p>
)}
</div>
</section>
<section id="education" className="border-line mx-auto max-w-6xl border-t px-4 py-14">
<SectionTitle>Education &amp;amp; Experience</SectionTitle>
<ol className="border-accent space-y-8 border-l-2 pl-6">
<li>
<p className="text-accent font-mono text-sm">Currently</p>
<h3 className="text-xl font-bold">Bachelor of Science in Information Technology</h3>
<p className="text-muted">Currently BSIT 3, Block 5</p>
</li>
<li>
<p className="text-accent font-mono text-sm">Learning</p>
<h3 className="text-xl font-bold">Web development</h3>
<p className="text-muted">Building responsive web applications and strengthening full-stack development skills.</p>
</li>
</ol>
</section>
<section id="contact" className="border-line mx-auto max-w-6xl border-t px-4 py-14">
<SectionTitle>Contact</SectionTitle>
<div className="grid gap-10 md:grid-cols-2">
<form className="space-y-5" onSubmit={submitContact}>
<div>
<label htmlFor="name" className="mb-1 block font-semibold">
Name
</label>
<input
id="name"
name="name"
type="text"
autoComplete="name"
required
className="border-line bg-surface w-full rounded-lg border px-3 py-2"
/>
</div>
<div>
<label htmlFor="email" className="mb-1 block font-semibold">
Email
</label>
<input
id="email"
name="email"
type="email"
autoComplete="email"
required
className="border-line bg-surface w-full rounded-lg border px-3 py-2"
/>
</div>
<div>
<label htmlFor="message" className="mb-1 block font-semibold">
Message
</label>
<textarea
id="message"
name="message"
rows={4}
minLength={10}
required
className="border-line bg-surface w-full rounded-lg border px-3 py-2"
/>
</div>
<button type="submit" className="bg-accent text-bg rounded-lg px-6 py-3 font-semibold hover:opacity-90">
Send message
</button>
<p className="font-semibold" role="status">
{contactStatus}
</p>
</form>
<div>
```

```
<h3 className="mb-4 text-xl font-bold">Find me online</h3>
<ul className="space-y-3">
<li>
<a className="hover:text-accent underline" href="mailto:your.email@example.com">
your.email@example.com
</a>
</li>
<li>
<a
className="hover:text-accent underline"
href="https://github.com/"
target="_blank"
rel="noopener noreferrer"
>
GitHub
</a>
</li>
<li>
<a
className="hover:text-accent underline"
href="https://www.linkedin.com/"
target="_blank"
rel="noopener noreferrer"
>
LinkedIn
</a>
</li>
</ul>
</div>
</div>
</section>
</main>
<PortfolioFooter />
</div>
</>
);
}
```

resources/js/types/index.ts

```
import { LucideIcon } from 'lucide-react';
export interface Auth {
user: User;
}
export interface BreadcrumbItem {
title: string;
href: string;
}
export interface NavGroup {
title: string;
items: NavItem[];
}
export interface NavItem {
title: string;
url: string;
icon?: LucideIcon | null;
isActive?: boolean;
}
export interface SharedData {
name: string;
quote: { message: string; author: string };
auth: Auth;
flash: { success?: string | null };
[key: string]: unknown;
}
export interface User {
id: number;
name: string;
email: string;
avatar?: string;
email_verified_at: string | null;
created_at: string;
updated_at: string;
[key: string]: unknown; // This allows for additional properties...
}
```

tests/Feature/ProjectManagementTest.php

```
<?php
use App\Models\Project;
use App\Models\User;
use Inertia\Testing\AssertableInertia as Assert;
test('the public portfolio displays saved projects', function () {
$project = Project::query()->create([
'title' => 'Portfolio redesign',
'description' => 'A responsive portfolio project.',
'category' => 'personal',
'technologies' => 'React, Laravel',
'url' => 'https://example.com',
]);
$this->withoutVite()->get('/')
->assertOk()
->assertInertia(fn (Assert $page) => $page
->component('welcome')
->has('projects', 1)
->where('projects.0.id', $project->id)
->where('projects.0.title', 'Portfolio redesign'));
});
test('guests cannot manage projects', function () {
$this->get('/projects')->assertRedirect('/login');
$this->get('/projects/create')->assertRedirect('/login');
$this->post('/projects')->assertRedirect('/login');
});
test('authenticated users can view project create and edit pages', function () {
$this->withoutVite()->actingAs(User::factory()->create());
$project = Project::query()->create([
'title' => 'A unique project',
'description' => 'Project description.',
'category' => 'school',
'technologies' => 'React',
]);
$this->get('/projects')
->assertOk()
->assertInertia(fn (Assert $page) => $page
->component('projects/index')
->has('projects', 1));
$this->get('/projects/create')->assertOk()->assertInertia(fn (Assert $page) => $page->component('projects/create'));
$this->get("/projects/{$project->id}/edit")
->assertOk()
->assertInertia(fn (Assert $page) => $page
->component('projects/edit')
->where('project.id', $project->id));
});
test('an authenticated user can create a project', function () {
$this->actingAs(User::factory()->create())
->post('/projects', [
'title' => 'Portfolio redesign',
'description' => 'A responsive portfolio project.',
'category' => 'personal',
'technologies' => 'React, Laravel',
'url' => 'https://example.com',
])
->assertRedirect('/projects')
->assertSessionHas('success', 'Project created.');
$this->assertDatabaseHas('projects', [
'title' => 'Portfolio redesign',
'category' => 'personal',
]);
});
test('all required project fields are validated when the form is empty', function () {
$this->actingAs(User::factory()->create())
->from('/projects')
->post('/projects', [])
->assertSessionHasErrors(['title', 'description', 'category', 'technologies']);
$this->assertDatabaseCount('projects', 0);
});
test('project text fields cannot exceed their maximum lengths', function () {
$this->actingAs(User::factory()->create())
->from('/projects/create')
->post('/projects', [
'title' => str_repeat('a', 121),
'description' => str_repeat('a', 1001),
'category' => 'school',
'technologies' => str_repeat('a', 256),
```

```
'url' => '',
])
->assertSessionHasErrors(['title', 'description', 'technologies'])
->assertSessionHas('_old_input.title', str_repeat('a', 121));
$this->assertDatabaseCount('projects', 0);
});
test('project category and URL must use supported formats', function () {
$this->actingAs(User::factory()->create())
->from('/projects/create')
->post('/projects', [
'title' => 'Invalid project',
'description' => 'A valid description.',
'category' => 'other',
'technologies' => 'React',
'url' => 'not-a-url',
])
->assertSessionHasErrors([
'category',
'url' => 'Enter a complete project URL, including https://.',
]);
});
test('project titles must be unique and show a custom validation message', function () {
$this->actingAs(User::factory()->create());
Project::query()->create([
'title' => 'Existing title',
'description' => 'Existing description.',
'category' => 'school',
'technologies' => 'React',
]);
$this->from('/projects/create')
->post('/projects', [
'title' => 'Existing title',
'description' => 'Another description.',
'category' => 'personal',
'technologies' => 'Laravel',
'url' => '',
])
->assertSessionHasErrors([
'title' => 'A project with this title already exists.',
]);
});
test('updating a project without changing its unique title is valid', function () {
$this->actingAs(User::factory()->create());
$project = Project::query()->create([
'title' => 'Keep this title',
'description' => 'Original description.',
'category' => 'school',
'technologies' => 'HTML, CSS',
]);
$this->put("/projects/{$project->id}", [
'title' => 'Keep this title',
'description' => 'Updated description.',
'category' => 'planned',
'technologies' => 'React',
'url' => '',
])->assertRedirect('/projects')->assertSessionHas('success', 'Project updated.');
$this->assertDatabaseHas('projects', [
'id' => $project->id,
'title' => 'Keep this title',
'description' => 'Updated description.',
]);
});
test('an authenticated user can update and delete a project', function () {
$this->actingAs(User::factory()->create());
$project = Project::query()->create([
'title' => 'Original project',
'description' => 'Original description.',
'category' => 'school',
'technologies' => 'HTML, CSS',
]);
$this->put("/projects/{$project->id}", [
'title' => 'Updated project',
'description' => 'Updated description.',
'category' => 'planned',
'technologies' => 'React',
'url' => '',
])->assertRedirect('/projects');
$this->assertDatabaseHas('projects', [
```

```
'id' => $project->id,
'title' => 'Updated project',
'category' => 'planned',
]);
$this->delete("/projects/{$project->id}")
->assertRedirect('/projects')
->assertSessionHas('success', 'Project deleted.');
$this->assertDatabaseMissing('projects', ['id' => $project->id]);
});
```